



Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR'S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

1	Introduction	1
1.1	Problem statement and societal relevance	1
1.2	Motivation: When does a simpler model pay off?	2
1.3	Research questions and objectives	2
1.4	Structure of the Thesis	2
2	Theoretical Background	3
2.1	Green AI and Red AI	3
2.1.1	The Rise of Red AI	3
2.1.2	Green AI as a Counter-Movement	4
2.1.3	Efficiency as a First-Class Metric	5
2.2	Energy and Carbon Emissions in Machine Learning	6
2.2.1	From Energy to Carbon Emissions	6
2.2.2	Measurement Approaches	7
2.2.3	CodeCarbon	8
2.3	Supervised Classification Algorithms	9
2.3.1	Logistic Regression	10
2.3.2	Decision Trees	11
2.3.3	Random Forest	12
2.3.4	Gradient Boosting and XGBoost	12
2.3.5	Multilayer Perceptron	12
2.3.6	Tree-based Models vs. Deep Learning on Tabular Data	14
2.3.7	Algorithmic Suitability for CPU vs. GPU	14
2.4	Dataset Scaling and Learning Curves	14
2.5	Evaluation Metrics	15
2.5.1	Confusion Matrix and Accuracy	15
2.5.2	Precision, Recall and F1-Score	15
2.5.3	Composite Efficiency Metrics	15
2.5.4	Cross-Validation	16
2.6	Hyperparameter Optimization	16
2.6.1	Grid Search and Random Search	16
2.6.2	Bayesian Optimization	16

2.6.3	Optuna as a Practical Framework	17
2.7	Hardware Considerations for Machine Learning	17
2.7.1	CPU Architecture for ML Workloads	17
2.7.2	GPU Computing and CUDA	17
2.8	Chapter Summary	18
3	Related Work	19
4	Experimental Design	20
4.1	Research Questions	20
4.1.1	Break-Even Analysis	21
4.2	Datasets	22
4.3	Models	22
4.4	Evaluation Metrics	23
4.5	Validation Strategy	24
4.6	Hyperparameter Tuning	25
5	Implementation and Measurement	26
5.1	Experimental Setup	26
5.1.1	Hardware Setup	26
5.1.2	Software Environment and Reproducibility	27
5.2	Dataset Integration	28
5.3	Model Implementations	29
5.3.1	Logistic Regression	30
5.3.2	Random Forest	30
5.3.3	XGBoost	31
5.3.4	Multilayer Perceptron	32
5.4	Hyperparameter Optimization	33
5.5	Emissions Measurement	35
5.5.1	CodeCarbon Integration	35
5.5.2	Hardware Measurement and Limitations	36
5.6	Inference Latency Measurement	37
5.7	Results Persistence and Orchestration	38
5.7.1	Results Schema	38
5.7.2	Experimental Orchestration	38
6	Results and Evaluation	40
7	Discussion	41
8	Conclusion and Outlook	42

Bibliography	43
List of Figures	47
List of Tables	48

Chapter 1

Introduction

1.1 Problem statement and societal relevance

The current trend in machine learning is moving towards increasingly large models and datasets [1]. Although deep learning approaches often achieve the highest predictive accuracy, their energy demand and associated CO_2 emissions increase disproportionately [2, 3]. In an era marked by strict sustainability guidelines and rising energy costs, a critical question arises: At what point is a computationally "simple" model ecologically more viable than a highly complex one?

The primary objective of this thesis is to determine whether and when it is worthwhile to deploy more complex models, considering not only their predictive accuracy but also their associated energy consumption. To achieve this, a Logistic Regression, a Random Forest, an XGBoost model and a Multilayer Perceptron are compared under equal conditions across three datasets of varying sizes. This comparative analysis aims to provide a clearer understanding of the thresholds at which simpler models represent the better choice. Additionally, the study investigates how modifying a Multilayer Perceptron architecture, specifically varying the number of neurons and hidden layers, directly impacts energy consumption and CO_2 emissions.

To address the problem statement and achieve the outlined objectives, this thesis investigates the overarching question: *How does the relationship between energy consumption and predictive accuracy change for Logistic Regression, Random Forest, XGBoost, and Multilayer Perceptron when model complexity and dataset size are systematically varied?*

Specifically, the following sub-questions are examined:

- How do energy consumption (in kWh) and CO_2 emissions change when scaling a deep learning architecture (variation of neuron count)?
- Under which conditions (dataset size, model complexity) does the resource consumption of complex models exceed the achievable accuracy gain compared to simpler algorithms?

- How does the energy consumption differ between CPU-based training (Random Forest) and GPU-accelerated training (XGBoost, MLP) for the same classification task?

1.2 Motivation: When does a simpler model pay off?

1.3 Research questions and objectives

1.4 Structure of the Thesis

The remainder of this thesis is structured as follows: Chapter 2 provides the theoretical background, distinguishing between Green AI and Red AI, and introducing the relevant sustainability and classification metrics. Chapter 3 details the methodology, including dataset selection and the experimental hardware setup. Chapter 4 outlines the implementation process, focusing on data preprocessing, model training, and the logging of resource consumption. The empirical results are analyzed in Chapter 5, evaluating both performance and resource utilization across scaling dataset sizes. Chapter 6 discusses the findings, highlighting the economic and environmental implications. Finally, Chapter 7 concludes the thesis and provides actionable recommendations for sustainable machine learning practices.

$$S = \frac{F_1}{1 + \lambda_1 \cdot t_{\text{scaled}} + \lambda_2 \cdot c_{\text{scaled}}}$$

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations required to understand the empirical study presented in the remainder of this thesis. It first frames the research within the broader debate between *Red AI* and *Green AI*, then formalizes the concepts of energy consumption and carbon emissions in machine learning. Subsequently, the relevant classification algorithms are described from a purely theoretical standpoint, followed by the metrics used to evaluate their predictive performance, the validation strategy, hyperparameter optimization techniques, and finally the hardware concepts that are central to comparing CPU- and GPU-based training. All implementation details are deliberately deferred to Section 5.3.

Diese Einleitung ist nur ein Platzhalter — am Ende des Chapters nochmal anpassen, sobald die finalen Sections feststehen.

2.1 Green AI and Red AI

The evaluation framework of this study assesses machine learning models not only by predictive performance, but explicitly by their ecological cost. This section introduces the conceptual distinction between Red AI and Green AI, which provides the vocabulary for the research questions and motivates the treatment of efficiency as a first-class evaluation criterion. The metrics used to operationalize ecological cost are defined in Section 2.5, and the corresponding measurement infrastructure is described in Section 2.2.

2.1.1 The Rise of Red AI

In recent years, a paradigm has emerged in Artificial Intelligence (AI) research that Schwartz et al. refer to as "Red AI" [1]. Red AI describes research approaches that primarily aim to improve prediction accuracy or related metrics through the massive use of computational power, data, and parameters, while largely disregarding the associated

infrastructural and energetic costs. In this paradigm, new state-of-the-art results are effectively "bought" by employing increasingly vast computational resources.

The background to this development is that the relationship between model performance and model complexity, whether measured by the number of parameters, the size of the dataset, or the number of experiments, is at best logarithmic [1]. This means that a linear increase in accuracy requires an exponentially larger model or an exponentially greater volume of training data. This dynamic is reflected in empirical scaling laws. Since 2012, the amount of compute used to train state-of-the-art AI models has been doubling approximately every 3.4 months [4].

This trend toward increasingly resource-intensive models is particularly evident in the evolution of modern deep learning architectures such as Transformers. Strubell et al. extensively document the massive computational requirements and corresponding energy consumption associated with training these deep neural networks, noting for instance that training a large Transformer model can require days of continuous computation across multiple GPUs [2]. This scaling trend is further amplified by the development of massive foundation models: OpenAI's GPT-2 already contains 1.5 billion parameters, while its successor GPT-3 scales up to 175 billion parameters [2]. Consequently, the continuous optimization of predictive accuracy objectively necessitates an exponentially growing demand for computational power and energy in modern AI research.

Definition Red AI nach Schwartz et al.: Forschung, die Genauigkeit primär durch immer mehr Compute, Daten und Parameter erkaufte. Skalierungsgesetz von Compute pro SOTA-Paper (Verdopplung alle 3.4 Monate, [5] bzw. analog). Kostentrends: Strubell et al. (Transformer-Training), GPT-Skalierung. Wichtig: nicht moralisieren, sondern Phänomen sachlich beschreiben.

2.1.2 Green AI as a Counter-Movement

In response to the resource-intensive trends described in the previous section, the concept of *Green AI* has emerged as a counter-movement within the machine learning community. Green AI is defined as research that yields novel results while explicitly accounting for computational costs, with the goal of reducing the resources required to produce them [1]. A core principle of this movement is the elevation of efficiency to a primary evaluation criterion, placing it alongside traditional metrics such as predictive accuracy [1].

A central demand of the Green AI movement is the establishment of standardized reporting practices. Specifically, researchers advocate that the computational work required to generate a result, quantified in floating-point operations (FLOPs), as well as the corresponding energy consumption and carbon emissions, should become standard practice in the machine learning community [1]. This transparency serves two purposes: it enables fair comparison between different approaches, and it establishes baselines

that future work can build upon.

An important distinction in this context is between Green AI and the concept of *AI for sustainability* (also referred to as *AI for Green*). AI for sustainability focuses on applying machine learning as a tool to address broader environmental challenges, such as optimizing industrial processes, managing forest fires, or supporting animal conservation efforts [1]. Green AI, by contrast, is exclusively concerned with reducing the environmental footprint of AI systems themselves [6]. These two goals are orthogonal: deploying AI for sustainable purposes does not make those AI systems inherently more efficient, and is therefore outside the scope of this thesis.

To support the adoption of Green AI principles in practice, several tools and frameworks have been developed to quantify the ecological impact of model training. Tools such as the ML CO₂ Impact Calculator [3] and the experiment-impact-tracker [7] allow practitioners to estimate carbon emissions based on hardware configuration, execution time, and the carbon intensity of the local electricity grid. At the organizational level, the Green Software Foundation has been working to formalize these efforts through the Software Carbon Intensity (SCI) specification, which aims to provide a standardized and comparable basis for sustainability metrics in software and AI systems [8].

2.1.3 Efficiency as a First-Class Metric

To counter the resource-intensive trends of Red AI, the Green AI movement advocates for making efficiency a primary evaluation criterion, alongside accuracy [1]. Evaluating models based on both performance and efficiency aligns incentives towards sustainable research and allows for a more rigorous cost-benefit analysis [7].

Realizing this in practice, however, requires defining how efficiency should be measured. The literature suggests several proxies, each with distinct trade-offs that determine their suitability for empirical comparison.

Floating-point operations (FLOPs) provide a hardware-independent estimate of computational work and allow algorithmic comparisons independent of the underlying infrastructure [1]. Their limitation is that FLOPs do not translate directly to energy consumption or execution time, as these depend on hardware architecture, memory access patterns, and parallelization [7]. Parameter count is another common proxy, but it can be misleading: models with similar parameter counts can perform vastly different amounts of computational work depending on their depth, width, or sparsity structure [1].

Training time is more intuitive and directly observable, but it is hardware-dependent and therefore difficult to compare across different computational environments [1]. Energy consumption in kWh and CO₂ equivalents (CO₂eq) sidestep this limitation by capturing the actual physical and ecological footprint of a run. Although these metrics are tied to a specific hardware configuration and electricity grid, they provide directly comparable sustainability figures when experiments are conducted under controlled

conditions [7], [9].

2.2 Energy and Carbon Emissions in Machine Learning

Operationalizing efficiency requires distinguishing two related but distinct quantities. Energy consumption is a hardware-level physical quantity, the product of power draw and time expressed in kilowatt-hours, where the actual power draw depends on hardware utilization and thermal design rather than being a fixed property of the components [7]. Carbon emissions then translate that energy into an ecological footprint, measured in carbon dioxide equivalents (CO₂eq), by multiplying it with the *carbon intensity* of the local electricity grid [3]. Two identical training runs can therefore produce the same energy consumption yet vastly different emissions, depending solely on the fuel mix of the grid supplying them [9]. In practice, empirical studies have shown that identical workloads can vary in their carbon footprint by a factor of 40 depending on geographic location [3], and can even fluctuate based on the time of day the models are trained [9].

2.2.1 From Energy to Carbon Emissions

To understand the ecological footprint of machine learning, it is first necessary to distinguish between power and energy. Power, measured in Watts (W) or Joules per second, represents the instantaneous rate at which electricity is drawn by a system. Energy, on the other hand, is the total quantity of electricity consumed over a specific duration, calculated as the product of power and time ($E = P \cdot t$), and is typically expressed in kilowatt-hours (kWh) or Joules [7].

The execution of machine learning models relies on a combination of hardware components, primarily the Central Processing Unit (CPU), Graphics Processing Unit (GPU), and dynamic random-access memory (DRAM) [7]. Modern AI workloads rely heavily on GPUs because their highly parallelized architectures provide significant computational acceleration compared to traditional CPUs [9]. However, this increased throughput comes at a steep energetic cost: while CPUs typically consume between 10W and 150W under load, GPUs are significantly more power-hungry, often drawing between 250W and 350W [9]. As a result, the GPU typically accounts for the vast majority, often nearly three-quarters, of the total electrical energy consumed during deep learning training [9]. Nevertheless, the energy drawn by the CPU and DRAM remains a non-negligible factor that must be recorded for comprehensive and accurate sustainability accounting [9].

When evaluating the power profile of these computing components, the Thermal Design Power (TDP) is a frequently referenced specification. TDP defines the maximum heat flow generated by a CPU or GPU that its cooling system is designed to dissipate,

serving as a theoretical indicator of the maximum power the component can draw [10]. Because direct, real-time power measurement can be complex, some simplified methodologies attempt to estimate total energy consumption by simply multiplying a component’s TDP by the total execution time of the model [10].

However, this approach is fundamentally flawed [7]. TDP represents an absolute peak thermal limit rather than an average operational consumption, and actual power draw fluctuates significantly based on hardware utilization, varying greatly between idle states and active load [7]. Empirical studies demonstrate that estimating efficiency solely via TDP and time, while neglecting real-time hardware utilization and memory effects, leads to severe over- or underestimations of a model’s true energy consumption and carbon footprint [7]. Consequently, reliable sustainability metrics require direct measurements of the hardware rather than abstract, TDP-based extrapolations.

Translating this measured energy consumption into a quantifiable environmental impact requires the concept of carbon intensity [3]. Carbon intensity represents the amount of greenhouse gas emissions produced per unit of electricity generated, expressed in grams of CO₂ equivalents per kilowatt-hour (gCO₂eq/kWh), and standardizes the global warming potential of various greenhouse gases into a single comparable value [7].

The operational carbon footprint of a machine learning model is derived by multiplying the total energy consumed (E , in kWh) by the carbon intensity of the local electricity grid (I), yielding $\text{CO}_2\text{eq} = E \cdot I$ [9]. Because different geographic regions rely on vastly different fuel mixes, from coal and natural gas to nuclear and renewables, carbon intensity varies drastically by location [7]. Empirical data illustrates this range clearly: training a model in Quebec, Canada, which is heavily reliant on hydroelectricity, faces a carbon intensity of approximately 20 gCO₂eq/kWh, whereas training the same model in a fossil-fuel-dependent region such as Iowa, USA, results in a carbon intensity of over 736 gCO₂eq/kWh [3].

Carbon intensity is not merely a static geographic property but also a temporally variable one [9]. The emissions factor of a grid can fluctuate significantly throughout the day based on the real-time availability of renewable sources, such as solar during daylight hours or wind at night [9]. Consequently, accurately evaluating the ecological footprint of a machine learning model requires integrating both the spatial location and the temporal characteristics of the local energy grid into the final CO₂eq conversion [9].

2.2.2 Measurement Approaches

Accurately evaluating the energy consumption of machine learning models relies on a spectrum of methodologies, broadly categorized into physical measurements, estimation models, and on-chip sensors [10]. The baseline for ground-truth measurement is the External Power Meter (EPM), which tracks power directly at the wall outlet or motherboard [10]. While EPMs capture the true consumption of the entire system, they are often costly to deploy at scale and cannot provide fine-grained decomposition to

isolate the energy used by specific software processes or individual hardware components [10].

To achieve component-level granularity without external hardware, many approaches utilize estimation models. These models calculate energy based on abstract hardware specifications, such as multiplying execution time by the Thermal Design Power (TDP) or counting Floating-Point Operations (FLOPs), or by leveraging Performance Monitoring Counters (PMCs) to track utilization rates [10]. However, as established in Section 2.2.1, relying solely on abstract analytical models or TDP often leads to severe inaccuracies under dynamic workloads, failing to capture the true, fluctuating real-time power draw of the hardware [7].

Consequently, modern sustainability accounting strongly prefers direct measurements obtained from vendor-specific on-chip sensors [7]. For GPUs, the industry standard is the NVIDIA Management Library (NVML), which polls instantaneous power consumption during execution [7]. For CPUs and DRAM, the standard interface is the Running Average Power Limit (RAPL), provided by Intel and AMD, which accesses hardware performance counters to deliver accurate, fine-grained energy measurements directly from the CPU package [10].

Despite its high accuracy, a significant practical limitation of RAPL is its strict dependency on the underlying operating system [7]. RAPL interfaces are primarily supported on Linux environments and often require elevated administrator privileges to access the hardware counters securely [7]. When direct sensor access is unavailable due to these constraints, software tracking tools often revert to problematic TDP-based estimation models as a fallback, which severely compromises measurement accuracy [10].

2.2.3 CodeCarbon

To practically implement the sustainability accounting principles advocated by the Green AI movement, researchers require standardized tools that bridge the gap between physical hardware metrics and environmental impacts. Initially, the machine learning community relied on a fragmented landscape of independent measurement approaches and preliminary software packages [11]. Early solutions largely consisted of web-based calculators, such as the ML CO₂ Impact calculator [3] and Green Algorithms (cite: Lannelongue et al.), which required practitioners to manually input hardware specifications, execution times, and locations to estimate carbon footprints.

While these online calculators successfully raised awareness, they lacked real-time monitoring capabilities and had to rely on broad estimations rather than ground-truth physical power draw [11]. To address this, several independent Python packages were developed to track energy dynamically during model training. Tools such as the experiment-impact-tracker [7], CarbonTracker (cite: Anthony et al.), and Energy Usage (cite: Lottick et al.) were introduced to directly poll hardware sensors during execution

[11]. However, the proliferation of tools with varying methodologies, carbon intensity data sources, and hardware support led to inconsistent reporting and discrepancies in carbon footprint measurements across the literature [11].

To unify these fragmented efforts into a reliable standard, the CodeCarbon initiative was established [11]. CodeCarbon is an open-source software library that evolved directly from previous tracking projects, serving as the official successor to the Energy Usage tool [10] and integrating the efforts of the ML CO₂ Impact project [11]. By providing a unified tracking interface, it enables practitioners to quantify the environmental cost of their algorithms and report these metrics transparently (cite: Schmidt et al.).

To evaluate a model’s energy consumption, the framework leverages the direct measurement approaches introduced in Section 2.2.2. Specifically, it interfaces with vendor-specific on-chip sensors, such as NVML for GPUs and RAPL for Intel and AMD processors, to collect fine-grained power draw measurements directly from the underlying hardware [10]. By periodically polling these hardware counters during execution, the tool computes the total electrical energy consumed by the computing components in kilowatt-hours [10].

Once the physical energy consumption is recorded, the framework translates it into greenhouse gas emissions using the conversion detailed in Section 2.2. The tool determines the carbon intensity of the local electricity grid based on the geographic location of the computing infrastructure, then multiplies the measured energy by this value to produce the final operational carbon footprint expressed in kilograms of CO₂-equivalents (kgCO₂eq) (cite: Schmidt et al.). By combining hardware-level energy tracking with grid-level carbon intensity data in a single standardized framework, CodeCarbon automates the translation from computational work to ecological impact [7].

2.3 Supervised Classification Algorithms

In machine learning, supervised classification is the task of inferring a predictive model from historical, annotated data in order to make accurate categorical predictions on new data [12]. Formally, given a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ consisting of n examples, where each $x_i \in \mathcal{X}$ represents a vector of input features and $y_i \in \mathcal{Y}$ represents the corresponding ground-truth label, the objective is to learn a mapping function $f : \mathcal{X} \rightarrow \mathcal{Y}$ that reliably predicts the target output for any given input [13]. This reliance on an explicitly provided label space distinguishes supervised learning from unsupervised learning, where algorithms discover hidden structures in unannotated data, and from reinforcement learning, where an agent learns to make sequential decisions by maximizing a reward signal within an interactive environment [12].

The fundamental objective when learning f is achieving strong *generalization*: the model must perform reliably on new, unseen data drawn from the same underlying

distribution rather than merely memorizing training inputs [14]. A central challenge during training is therefore finding the optimal balance to prevent *underfitting*, where the model is too simple to capture the true data patterns, and *overfitting*. Overfitting occurs when a model excessively adapts to the specific “sampling noise” present in the training data, fitting it so closely that its predictive performance on new, unseen data actively degrades [14], [15].

The structure of the label space $\mathcal{Y}$ determines the nature of the classification task. In binary classification, $\mathcal{Y} = \{0, 1\}$; when more than two categories are involved it becomes a multiclass problem with $\mathcal{Y} = \{1, \dots, C\}$ [13]. Multiclass problems are typically handled via One-vs-Rest strategies or by extending binary formulations with a Softmax output that produces a normalized probability distribution across all C classes.

To optimize f during training, a loss function $\mathcal{L}(y, f(x))$ quantifies the discrepancy between predicted outputs and true labels [15]. For classification, the standard objective is Cross-Entropy Loss. In the binary case this reduces to the negative binomial log-likelihood; for multiclass problems it generalizes to Categorical Cross-Entropy:

$$\mathcal{L} = - \sum_{k=1}^C y_k \log(p_k(x)) \quad (2.1)$$

where y_k is the binary indicator for the true class k and $p_k(x)$ is the model’s predicted probability for that class [15]. Cross-Entropy is preferred over Mean Squared Error for classification because it produces steeper, more informative gradients when a confident prediction is wrong, and provides a direct probabilistic interpretation of the model output. This foundational loss objective drives the gradient-based optimization for both the deep learning and advanced ensemble architectures discussed in the following subsections.

2.3.1 Logistic Regression

Logistic regression, formalized as a statistical classification method by Cox [16], is a parametric linear model that remains a standard baseline for binary classification tasks where interpretability and computational efficiency are paramount.

For a binary classification task ($\mathcal{Y} = \{0, 1\}$), logistic regression predicts the probability that an instance x belongs to the positive class. To ensure the output is bounded strictly between 0 and 1, the linear combination of input features and learned weights w with bias b is passed through the logistic sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, giving:

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}} \quad (2.2)$$

This functional form stems from modeling the log-odds (logit) as a linear function of the inputs: $\ln\left(\frac{P(y=1|x)}{P(y=0|x)}\right) = w^\top x + b$ [17].

The optimal parameters are estimated via Maximum Likelihood Estimation (MLE), which in practice is equivalent to minimizing the binary Cross-Entropy Loss over the training set [17]. Since no closed-form solution exists, iterative numerical solvers are required. Standard approaches include second-order methods such as Newton-Conjugate Gradient (Newton-CG) and Limited-memory BFGS (L-BFGS), as well as first-order methods such as Stochastic Average Gradient (SAG) descent.

Logistic regression extends naturally to multiclass problems ($\mathcal{Y} = \{1, \dots, C\}$) via a One-vs-Rest (OvR) strategy or by replacing the sigmoid with a Softmax function, yielding Multinomial Logistic Regression.

A key advantage of logistic regression is its interpretability: each feature weight directly represents its directional influence on the log-odds of the prediction. It also trains quickly and is robust to overfitting on simple datasets. Its fundamental limitation is linearity. Without manual feature engineering, the model cannot capture non-linear patterns or feature interactions, making it considerably less expressive than the tree-based and deep learning architectures described in the following subsections.

2.3.2 Decision Trees

Decision trees are a family of non-parametric, non-linear classification models. Unlike logistic regression, which assumes a linear decision boundary, decision trees partition the input feature space into a set of disjoint, piece-wise constant regions [15], [18]. This property allows them to naturally learn highly irregular patterns and high-frequency functions that linear models and even standard neural networks often struggle to fit [13].

Structurally, a decision tree is constructed from the training data by mapping these disjoint regions to terminal leaf nodes [15]. The internal nodes represent binary tests on input features, while the terminal leaf nodes represent the final predicted class labels. To classify an input vector, the algorithm routes the sample down the tree by applying a sequence of these binary tests until a leaf is reached.

During training, the tree is typically grown using the CART (Classification and Regression Trees) methodology [15], [19]. This employs a recursive, greedy splitting procedure: at each internal node, the algorithm evaluates possible split points across all features and greedily selects the one that maximizes class homogeneity in the resulting child nodes [20]. For classification tasks, this homogeneity is traditionally measured using criteria such as Gini Impurity or Information Gain (Entropy).

One of the primary advantages of decision trees is their high interpretability; the sequential decision rules can be explicitly visualized and understood. Furthermore, they are inherently capable of capturing complex, non-linear relationships and interaction effects between variables without requiring extensive feature engineering or data scaling.

However, single decision trees suffer from severe theoretical limitations. If allowed to grow to their maximum depth without regularization, they are highly prone to

overfitting: they excessively adapt to the specific “sampling noise” present in the training data, fitting it so closely that their predictive performance on new, unseen data actively degrades [15]. Additionally, single trees are notoriously unstable, meaning a minor perturbation in the training dataset can cause a drastically different tree structure to be generated. This inherent instability, high variance, and overfitting risk serve as the primary motivation for aggregating multiple trees into ensemble methods such as Random Forests and Gradient Boosting [19].

2.3.3 Random Forest

Bagging-Prinzip nach [19]: n Bootstrap-Samples $\rightarrow n$ Bäume $\rightarrow$ Majority Vote / Mittelwert. Zusätzliche Randomisierung über zufällige Feature-Subsets pro Split (mtry / max_features). Warum das die Varianz reduziert (Bias-Varianz-Zerlegung kurz). Begriff Out-of-Bag (OOB) Error. Parallelisierbarkeit über Bäume hinweg $\rightarrow$ Ankerpunkt für CPU-Multi-Threading-Diskussion in Section 2.7. Skalierungsverhalten: superlinear in n bei tiefen Bäumen — Begründung, warum RF auf HIGGS in Section 5.3.2 ausgeschlossen wurde, schon hier theoretisch andeuten.

shouldnt grinsztajn be cited more often seeing as this name of the paper aligns best with the decision tree subsection

2.3.4 Gradient Boosting and XGBoost

Boosting-Idee: sequentiell schwache Lerner addieren, jeder fittet Residuen / Gradienten des bisherigen Ensembles. Formal: Funktionsraumgradientenabstieg [15]. Übergang zu XGBoost [20]: 2nd-Order-Gradienten (Hessian), Regularisierungsterm im Loss ($\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$), Shrinkage (learning rate), Column Subsampling. Histogram-basierte Split-Findung (gpu_hist) als Brücke zu GPU-Beschleunigung — wichtig für RQ "CPU vs. GPU".

2.3.5 Multilayer Perceptron

Section-Einleitung: kurz historisch (Perceptron [21], dann Mehrschicht via Backprop [22]). Danach modular in Subsubsections — der derzeit in Section 5.3.4 stehende theoretische Block ("BatchNorm normalisiert Aktivierungen über den Mini-Batch ...") soll hierhin verschoben werden, dort nur noch die Implementation übrig lassen.

Perceptron and Feedforward Architecture

Einzelner Perceptron als gewichtete Summe + Aktivierung. Stack zu Schichten: $h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)})$. Universal Approximation Theorem in 1–2 Sätzen erwähnen. Architekturparameter: Anzahl Hidden Layers, Neuronen pro Layer — direkter Bezug zu RQ "Skalierung der Architektur".

Activation Functions

Sigmoid, Tanh, ReLU vergleichen. Vanishing-Gradient-Problem von Sigmoid/Tanh, daher Dominanz von ReLU [23]. Eigenschaften ReLU: $\max(0, x)$, sparse Aktivierungen, billiger Gradient. Kurze Erwähnung von Varianten (Leaky ReLU, GELU) — aber begründen, warum in dieser Arbeit Standard-ReLU genutzt wird.

Backpropagation and Gradient Descent

Chain Rule für Gradienten durch das Netz (Backpropagation). Stochastic Gradient Descent (SGD) → Mini-Batch SGD → Adam [24] mit adaptiven Momenten. Formel Adam-Update kompakt. Konzept Learning Rate als wichtigster Hyperparameter (motiviert spätere Tuning-Suchräume).

Batch Normalization

Inhalt aus Section 5.3.4 hierher verschieben: Normalisierung der Aktivierungen pro Mini-Batch zu Mittelwert 0 / Varianz 1, dann affine Transformation mit lernbaren γ, β . Effekt: Reduktion des Internal Covariate Shift, schnellere Konvergenz, leichte Regularisierung [25].

Dropout

Inhalt aus Section 5.3.4 hierher verschieben: zufälliges Deaktivieren von Neuronen mit Wahrscheinlichkeit p während des Trainings, volle Aktivierung beim Inference (skaliert). Interpretation als implizites Ensemble [14]. Warum BN + Dropout zusammen funktionieren (mit Vorsicht — bekannte Wechselwirkungen kurz erwähnen).

Early Stopping as Regularization

Konzept: Training abbrechen, sobald Validation-Loss nicht mehr sinkt. Theoretische Begründung als impliziter Regularisierer (begrenzt effektive Modellkapazität). Patience-Parameter erklären. Querverweis: konkrete Implementation in Section 5.3.4.

2.3.6 Tree-based Models vs. Deep Learning on Tabular Data

Neural networks have achieved state-of-the-art results on image and text data, but their advantage does not straightforwardly transfer to tabular data. Recent benchmark studies show that tree-based methods consistently match or outperform deep learning models on tabular datasets, even after extensive tuning of the neural architectures [13], [18]. A key reason is that neural networks exhibit an inductive bias toward smooth, continuous functions, whereas tabular features often exhibit irregular, non-monotonic interactions that decision tree splits capture naturally without requiring the network to learn them implicitly [18].

This has a direct implication for the energy efficiency analysis in this thesis. On the tabular datasets used here, the MLP may require substantially more hyperparameter search and training compute to reach the accuracy levels achieved by XGBoost or Random Forest, with no guarantee of matching their predictive performance. The inherent difficulty of fitting a neural network to tabular data can therefore inflate its energy cost well beyond what its architectural throughput alone would suggest.

2.3.7 Algorithmic Suitability for CPU vs. GPU

Zusammenfassend: welche Algorithmen profitieren wovon? Logistic Regression: meist CPU (Datentransfer-Overhead lohnt sich nicht). Random Forest: CPU (irreguläre Tree-Traversal). XGBoost: beides möglich (`hist` auf CPU, `gpu_hist` auf GPU) — dieser Vergleich ist Kern einer RQ. MLP: GPU klar überlegen ab nicht-trivialer Größe. Energiebetrachtung anschließen: GPU hat höhere Peak-Leistung, aber bei passenden Workloads deutlich mehr Sample/J $\rightarrow$ bessere Energieeffizienz, NICHT zwangsläufig weniger Energieverbrauch absolut. Diese Theorie wird in Chapter 6 empirisch geprüft.

2.4 Dataset Scaling and Learning Curves

A learning curve plots a model’s predictive performance as a function of the number of training samples n . The general pattern observed across model families is consistent: performance improves rapidly when training data is scarce, then exhibits diminishing returns as n grows [26]. This relationship is often approximated as a power law $P(n) \propto n^\alpha$ with $\alpha \in (0, 1)$, meaning that each successive doubling of the dataset yields a smaller absolute gain in performance than the previous one [27].

This diminishing-returns property has a direct implication for computational cost. Training time and energy consumption scale roughly linearly with dataset size for most algorithms: processing twice as many samples requires approximately twice the compute. The combination of linear cost growth and sub-linear performance growth creates a fundamental tension — achieving a marginal gain in accuracy at the upper end of a

learning curve demands a disproportionately large investment in additional data, time, and energy [1].

This tension is central to the research questions of this thesis. By training and evaluating each algorithm at multiple dataset scales, the empirical study can locate where a model’s learning curve begins to flatten relative to its energy consumption, and determine whether the performance gain from scaling up justifies the additional ecological cost. The specific dataset sizes examined in this study are described in Section 4.2.

2.5 Evaluation Metrics

Section-Einleitung: drei Metrik-Familien — (i) Klassifikationsgüte, (ii) Ressourcenverbrauch, (iii) zusammengesetzte Effizienzmetriken. Wird aus Section 4.4 referenziert.

2.5.1 Confusion Matrix and Accuracy

Confusion Matrix für binär (TP, TN, FP, FN) und Erweiterung auf multiklass. Accuracy = $(TP + TN)/N$ bzw. Anteil korrekter Vorhersagen. Limitation bei Class Imbalance demonstrieren mit kleinem Beispiel (z.B. 99 % Mehrheitsklasse → 99 % Accuracy mit trivialem Klassifikator) — motiviert F1.

2.5.2 Precision, Recall and F1-Score

Definitionen: Precision = $TP/(TP + FP)$, Recall = $TP/(TP + FN)$, F1 = harmonisches Mittel. Warum harmonisches statt arithmetisches Mittel (penalisiert Extremwerte). Macro / Micro / Weighted F1 unterscheiden — Weighted F1 = pro Klasse gewichtet mit Support, nutzt diese Arbeit. Begründung für Weighted F1 bei den hier verwendeten Datasets (Credit ist imbalanced, HIGGS leicht imbalanced).

2.5.3 Composite Efficiency Metrics

Kurz die Ressourcenmetriken einführen: Trainingszeit (s), Energie (kWh), CO₂eq (kg), Inference-Latenz (s/Sample). Querverweis auf Section 2.2 für Definitionen. Danach: generelle Idee zusammengesetzter Metriken — ein Score, der Performance gegen Kosten abwägt. Beispiele aus der Literatur: Energy-Efficiency Ratio, Performance-per-Watt, FLOPs-per-Sample. Hier theoretischer Hintergrund — die konkrete EWF1-Definition für diese Arbeit steht in Section 4.4, hier nur die methodische Grundlage und Schwächen solcher Composite-Metriken (Gewichtungswillkür, Hardware-Abhängigkeit) diskutieren.

2.5.4 Cross-Validation

Als Einstieg kurz Hold-Out (80/20-Split) beschreiben: einfach, aber hohe Varianz bei kleinen Datasets — motiviert K-Fold. K-Fold-Algorithmus: Daten in k Folds, k -mal trainieren, jeweils ein Fold als Test. Mittelwert über Folds als Performance-Schätzer, Standardabweichung als Stabilitätsmaß. Trade-off k : kleines $k \rightarrow$ schneller, höherer Bias; großes k (LOOCV) $\rightarrow$ niedriger Bias, sehr teuer. $k = 5$ und $k = 10$ als Standardwerte [27]. Rechtfertigung für $k = 5$ in dieser Arbeit verlinken auf Section 4.5. Abschließend: Stratifizierung — Klassenverhältnisse in jedem Fold erhalten, wichtig bei imbalanced Datasets (Credit, HIGGS).

2.6 Hyperparameter Optimization

Section-Einleitung: kurze Abgrenzung Parameter (während Training gelernt) vs. Hyperparameter (vor Training festgelegt: Lernrate, Tiefe, Anzahl Bäume). Suchraum als Cartesian Product der HP-Wertebereiche. Verweis auf Section 4.6 für die konkrete Begründung der Methodenwahl in dieser Arbeit.

2.6.1 Grid Search and Random Search

Grid Search: erschöpfende Suche über vorgegebenes Gitter. Skaliert exponentiell mit Anzahl HP — konkretes Rechenbeispiel passt zu Section 4.6 ($5 \text{ HPs} \times 10 \text{ Werte} \times 4 \text{ Modelle} \times 3 \text{ Datasets} = 150,000 \text{ Evaluationen}$). Random Search nach [28]: zufällige Stichprobe aus Suchraum, in höherdimensionalen Räumen oft effizienter als Grid. Beide Verfahren *informationslos* (lernen nichts aus vergangenen Trials) — das motiviert Bayes.

2.6.2 Bayesian Optimization

Sequentielles modellbasiertes Optimieren: Surrogatmodell (z.B. Gaussian Process oder TPE) approximiert die Objective, Acquisition Function (Expected Improvement, UCB) wählt nächsten Punkt. Exploration vs. Exploitation. TPE (Tree-structured Parzen Estimator) nach [29] kurz beschreiben — modelliert $p(x|y)$ statt $p(y|x)$, gut für mixed/conditional search spaces. Komplexitätsvorteil ggü. Grid Search bei vielen HPs.

2.6.3 Optuna as a Practical Framework

Optuna [30] als define-by-run-Framework: Suchraum wird im Objective-Code dynamisch konstruiert (statt vorab als Config). Standardsampler ist TPE. Pruner-Konzept (frühes Abbrechen schlechter Trials, z.B. MedianPruner) — nur theoretisch, ob diese Arbeit Pruning nutzt steht in Section 5.4. Reproducibility über Sampler-Seed. KEINE Code-Snippets hier — die gehören in Chapter 5. Zum Abschluss: ökologischer Aspekt von HPO — Gesamtkosten eines Ergebnisses wachsen linear mit der Anzahl der Trials [1]. Grid Search verschwendet Trials in irrelevanten Regionen des Suchraums; Bayesian Optimization (und damit Optuna) ist daher nicht nur mathematisch effizienter, sondern auch eine Green-AI-Strategie: weniger Trials = weniger Energie = kleinerer CO₂-Fußabdruck des gesamten Entwicklungsprozesses.

2.7 Hardware Considerations for Machine Learning

Section-Einleitung: motiviert die CPU-vs-GPU-Forschungsfrage aus theoretischer Sicht. Konkrete verwendete Hardware steht in Section 5.1.1, hier nur die Konzepte.

2.7.1 CPU Architecture for ML Workloads

CPU-Eigenschaften: wenige starke Cores, hohe Single-Thread-Leistung, große Caches, latenzoptimiert. Multi-Threading via OpenMP / Intel MKL für Matrixoperationen. Warum CPUs für RF-Bagging gut geeignet sind (Bäume sind unabhängig parallelisierbar, aber jeder Baum selbst ist sequentiell mit irregulären Speicherzugriffen).

2.7.2 GPU Computing and CUDA

GPU-Architektur: tausende einfache Cores, SIMT-Execution, durchsatzoptimiert. CUDA als NVIDIA-Programmiermodell. Warum Matrixmultiplikation (und damit NN-Training) ideal für GPUs ist: dichte parallele Operationen mit regelmäßigem Speichermuster. Memory-Hierarchie GPU (HBM/GDDR vs. Shared Memory) kurz erwähnen, nur soweit für Verständnis nötig.

2.8 Chapter Summary

Kurzes Fazit (max. ein Absatz): die wichtigsten theoretischen Bausteine für die folgenden Chapters sind benannt — Green-AI-Framing, Energie-/Emissions-Definitionen, Algorithmen-Theorie, Metriken, Validation, HPO und Hardware. Brücke zu Chapter 3 (Related Work) und Chapter 4 (Experimental Design).

Chapter 3

Related Work

Chapter 4

Experimental Design

This chapter outlines the experimental design used to evaluate the ecological efficiency of different machine learning classification algorithms. It describes the datasets, models, evaluation metrics, hyperparameter tuning and measurement setup that form the basis of this empirical analysis.

4.1 Research Questions

The main research question that this study aims to answer is: *How does the trade-off between energy consumption and predictive performance vary across Random Forest, XGBoost and MLP models when the complexity of the models and the size of the dataset are systematically varied?* The following sub-questions will also be addressed in the course of this thesis:

- Under what conditions do the resource consumption of more complex models exceed the achievable accuracy gain over simpler baselines?
- How does energy consumption and CO₂ output scale with increasing neural network complexity (number of layers and neurons)?
- How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?
- How does reducing the number of training instances on large datasets affect model accuracy and energy consumption across different algorithms?
- How accurate are software-based energy estimation tools compared to direct hardware sensor measurements, and how does this affect reported CO₂ emissions?

Letzte Unterfrage in der Diskussion ausführlicher behandeln: CodeCarbon unterschätzt CPU-Verbrauch empirisch um Faktor 2–9× (gemessen via LibreHardwareMonitor CPU Package Power Sensor). Begründung: TDP-basierte Schätzung skaliert linear mit CPU-Auslastung, bildet aber weder Boost-Verhalten noch Uncore-Komponenten ab. Konkrete Messwerte als Beleg vorhanden (z.B. LogReg Credit: 6.6 W CodeCarbon vs. 57.8 W HardwareMonitor; LogReg Higgs: Faktor $\sim 2.3\times$).

RQ4 (Scaling): Kurz das experimentelle Design für den Subset-Run beschreiben. Die Kernidee: statt verschiedener Datasets wird *derselbe* Datensatz (HIGGS) in fünf Größen aufgeteilt (100k, 500k, 1M, 5M, 11M Instanzen via TEST_ROWS Umgebungsvariable). Alle Modelle außer Random Forest werden mit denselben Hyperparametern (aus dem HIGGS-Tuning) auf jeder Größe trainiert. So wird der Dataset-Größen-Effekt isoliert, ohne dass unterschiedliche Hyperparameter oder Feature-Räume den Vergleich verfälschen. Ergebnis: CO₂, Trainingszeit und F1 als Funktion von n_rows für alle Modelle.

4.1.1 Break-Even Analysis

To determine at which dataset size GPU training becomes more energy-efficient than CPU training for XGBoost, a break-even analysis was designed as part of this study. The analysis is based on nine HIGGS subsets ranging from 1,000 to 11,000,000 instances, using identical hyperparameters for both devices. This controlled setup isolates the effect of dataset size without confounding factors from different feature spaces or hyperparameter configurations.

The relationship between dataset size and CO₂ emissions is modelled as a power law, fitted separately for CPU and GPU by applying a linear regression to $\log_{10}(\text{nrows})$ versus $\log_{10}(\text{CO}_2)$. A log-log fit is used because emissions scale super-linearly with data size, and a linear model would systematically misrepresent the relationship. The break-even point n_{BE} is then derived analytically as the intersection of the two fitted lines:

$$\log_{10}(n_{BE}) = \frac{b_{\text{GPU}} - b_{\text{CPU}}}{a_{\text{CPU}} - a_{\text{GPU}}} \quad (4.1)$$

where a and b are the slope and intercept of each fit. Below n_{BE} , CPU training is more energy-efficient; above it, GPU training is. The results are presented in Chapter 6.

To answer these questions multiple classification algorithms and datasets were used in the experimental phase of this empirical analysis.

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training [10]. To evaluate how this relationship varies across different scales, three classification datasets were selected that span several orders of magnitude in size. Classification was chosen as the task type since it is well-established across all selected models and allows for direct comparison of predictive performance via standardized metrics such as accuracy and F1-score. The datasets are summarized in Table 4.1.

Table 4.1: Overview of datasets used in this analysis

Dataset	Instances	Features	Classes	Scale
Wine [31]	178	13	3	Small
Default of Credit Card Clients [32]	30,000	23	2	Medium
HIGGS [33]	11,000,000	28	2	Large

The *Wine* dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The *Default of Credit Card Clients* (hereafter: *Credit*) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The *HIGGS* dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [34]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

These three datasets vary significantly in size, enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity.

4.3 Models

Selecting models of varying complexity is central to answering the research questions of this study. A model that achieves competitive accuracy with significantly lower energy consumption represents a more ecologically efficient choice, but this trade-off can only be quantified if the comparison spans a meaningful range of complexity. For this reason, five classification models were selected, ranging from a linear baseline to tree-based ensemble methods and a neural network, covering low, medium, and high computational demand respectively.

Logistic Regression serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive accuracy, making it well

Table 4.2: Overview of classification models used in this study

Model	Type	Device	Role
Logistic Regression [16]	Linear	CPU	Baseline
Random Forest [19]	Ensemble	CPU	Mid-complexity
XGBoost [20]	Gradient Boosting	CPU & GPU	Mid-complexity
Multilayer Perceptron	Neural Network	GPU	High-complexity

suiting to quantifying whether the additional resource consumption of more complex models yields a meaningful gain in predictive performance.

Random Forest was selected as a mid-complexity representative of tree-based ensemble methods, as extensively described in Section 2.x. Relevant to this study, Random Forest does not support GPU-accelerated training and is therefore run exclusively on the CPU.

Building on the tree-based approach, XGBoost was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to one of the research questions of this study.

The MLP was selected as the most complex model, representing the deep learning category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption. A more detailed description of the MLP architecture is provided in Section 2.x.

Together, the selected models cover a broad spectrum of algorithmic complexity, from a linear baseline to gradient boosting and deep learning, enabling a meaningful comparison of both predictive performance and energy efficiency across fundamentally different model types. The following section defines the metrics used to evaluate these dimensions.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and resource consumption. Accuracy and weighted F1-Score were selected to measure predictive performance, as they provide a standardized basis for comparing models across datasets of varying class distributions, as further described in Section 2.5. Energy consumption in kWh and CO_2 emissions in kg were chosen as the primary resource metrics, directly reflecting the ecological cost of model training. It should be noted that all experiments were conducted on a single hardware setup, which is described in detail in Section 5.1.1.

Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated predictions can often rival or even exceed the initial training costs [35]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in resource consumption.

Inspired by the efficiency framing proposed in Schwartz et al., a custom composite metric was developed for this study, named the *Efficiency-Weighted F1* (EWF1), defined as:

$$\text{EWF1} = \frac{F_1}{1 + \lambda_1 \cdot \hat{t} + \lambda_2 \cdot \hat{c}} \quad (4.2)$$

where $\hat{t}$ and $\hat{c}$ denote the min-max normalized training time and CO2 emissions per dataset respectively. The weights were set to $\lambda_1 = 0.5$ and $\lambda_2 = 1.0$, assigning greater importance to carbon emissions than training time, reflecting the ecological focus of this study. Higher scores indicate a more favorable trade-off between predictive performance and resource consumption.

It should be noted that both $\hat{t}$ and $\hat{c}$ are inherently hardware- and location-dependent, as training time varies with the underlying hardware configuration and CO2 emissions are tied to the carbon intensity of the local electricity grid [1]. Consequently, EWF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other hardware or energy infrastructures.

4.5 Validation Strategy

To obtain reliable and generalizable performance estimates, a cross-validation strategy was employed rather than a single train-test split, as described in Section 2.x.

Specifically, K-Fold cross-validation was applied with $k = 5$ folds, a value widely recommended in the literature as a practical balance between computational cost and estimation stability [27]. Each model is trained and evaluated five times on non-overlapping subsets of the data, and the final performance score is averaged across all folds. This approach is consistent across all models and datasets, ensuring comparability of results.

To ensure reproducibility, a fixed random state was used for all cross-validation splits, model initialization, and data shuffling. The specific value and its technical implementation are described in Section 5.1.2.

4.6 Hyperparameter Tuning

To ensure a fair comparison across all models, hyperparameter tuning was applied to each model prior to evaluation. Without tuning, default parameters tend to favor certain model families over others, potentially skewing the comparison in terms of both predictive performance and resource consumption [36].

Hyperparameter optimization was performed using Optuna [30], an optimization framework based on Bayesian search (see Section 2.x).

Hier in 02 die genaue Erklärung

Grid search was considered but deemed infeasible given the scale of experiments: assuming five hyperparameters per model with ten candidate values each, evaluated across three datasets and four models, a full grid search would require approximately 150,000 evaluations. Optuna was therefore configured to run 50 trials per model per dataset, striking a balance between tuning thoroughness and resource consumption, the latter being a relevant factor given that tuning runs contribute to the overall energy budget of this study, as described in Section 4.4.

Weighted F1-score was used as the optimization objective, as it accounts for class imbalance more robustly than accuracy, providing a more reliable estimate of model quality across datasets with varying class distributions.

Tuning was performed separately for each dataset, as the datasets differ substantially in size, feature space, and class distribution. A globally tuned model would not represent optimal performance on any individual dataset and would therefore introduce bias into the comparison.

Chapter 5

Implementation and Measurement

This chapter describes how the experimental pipeline was built and how measurements were collected. It covers the hardware and software environment, the data loading architecture, each model implementation, the hyperparameter tuning setup, and the emissions measurement infrastructure including the custom CPU power monitoring solution.

5.1 Experimental Setup

5.1.1 Hardware Setup

A critical decision in the experimental design was the selection of the computing platform on which all experiments would be conducted. Two options were considered: the university’s shared computing infrastructure or a local machine. The latter was ultimately chosen, as it provides several key advantages for this study. First, it ensures complete reproducibility, since all experiments are executed on identical hardware configuration, eliminating interfering variables that could arise from shared resource conflicts or system variability. Second, it provides consistent availability and direct control over system configuration, which is particularly important given the extended runtime of the large-scale experiments (particularly on the 11M-instance HIGGS dataset). Third, local execution facilitates fine-grained performance monitoring and measurement of energy consumption, a primary objective of this study.

The hardware platform selected for all experiments is a consumer-grade desktop system with the following specifications: an NVIDIA RTX 4070 graphics processing unit (12 GB GDDR6 memory, 5,888 CUDA cores), an AMD Ryzen 7 5700X processor (8 cores / 16 threads, base clock 3.6 GHz), and 32 GB of DDR4 RAM. The system runs Windows 11 with all code implemented in Python 3.13.0.

The choice of hardware reflects a deliberate trade-off between experimental scope and practical feasibility. The RTX 4070 represents a mid-to-high-end consumer GPU with sufficient compute capability to execute GPU-accelerated training for XGBoost and MLP

Table 5.1: Hardware specification of the experimental system

Component	Specification
CPU	AMD Ryzen 7 5700X (8 cores / 16 threads, 3.6 GHz base)
GPU	NVIDIA RTX 4070 (12 GB GDDR6, 5,888 CUDA cores)
RAM	32 GB DDR4
Operating System	Windows 11

models in reasonable time, while remaining accessible to many practitioners. The Ryzen 7 5700X provides adequate CPU performance for baseline models (Logistic Regression, Random Forest) and CPU-based XGBoost training. Together, this configuration enables direct comparison of CPU- and GPU-accelerated training on the same platform, which is central to answering one of the research questions (*How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*).

It is important to note that energy consumption and CO2 emissions measured in this study are specific to this hardware configuration and the local electricity grid, and may not directly generalize to other systems or geographic regions. However, the relative performance comparisons (e.g., the energy efficiency ranking of different algorithms) are more likely to generalize, as they reflect algorithmic properties rather than hardware specifics alone.

5.1.2 Software Environment and Reproducibility

All experiments were implemented in Python 3.13.0. The main libraries used are described below.

Scikit-learn [37] served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as `KFold` and `cross_validate` for cross-validation, `StandardScaler` for feature normalization, `make_pipeline` for chaining preprocessing and model steps, and `f1_score` and `make_scorer` for evaluation metrics.

The XGBoost library [20] was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [38] was used as the deep learning backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [39] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Hyperparameter tuning for all models except Logistic Regression was performed using Optuna, as described in Section 4.6.

Carbon emissions and energy consumption were tracked using CodeCarbon [40], which is described in detail in Section 5.5.

To ensure reproducibility, a fixed random seed of 42 was used for all data splits, cross-validation folds, and model initialization. A `requirements.txt` file is provided

so that the exact library versions used in this study can be reproduced.

The implementation code was developed with the assistance of Claude Code **anthropicClaudeCode2025**, an AI-powered development tool by Anthropic. All generated code was reviewed, tested, and integrated by the author.

To ensure fully reproducible results across all experiments, a global constant `RANDOM_STATE` with a value of 42 is used in every model script. It is passed to the `KFold` splitter and all model constructors. For the MLP, the seed is additionally set via `torch.manual_seed(RANDOM_STATE)` to guarantee deterministic weight initialization.

The evaluation relies on `KFold` with five splits and shuffling enabled, executed through `cross_validate()`. Performance is measured using a scoring dictionary that tracks both accuracy and weighted F1 score via `make_scorer(f1_score, average="weighted")`. Final scores are the mean across all five folds.

5.2 Dataset Integration

The three datasets used in this study, *Wine*, *Credit*, and *HIGGS*, were introduced in Section 4.2. This section describes how they are integrated into the software architecture, covering the central configuration file, the data loading pipeline, and dataset-specific constraints.

To ensure a consistent foundation across all model scripts, a central configuration file (`config.py`) was created. It serves as a single source of truth for all dataset-related settings, so that each script accesses the same parameters without redundant definitions.

For each dataset, the configuration stores the file path, the name of the target column, and an optional row limit (`nrows`). The row limit was particularly relevant for the *HIGGS* dataset, which contains 11 million instances, allowing experiments to be run on smaller subsets where needed. Beyond these common fields, the configuration also captures dataset-specific properties. For example, the *Credit* dataset includes a header row that must be skipped during loading, and the *Wine* dataset does not contain column names in the source file, which are therefore defined in the configuration directly. Additionally, a label offset is applied to the *Wine* target column, as its class labels are encoded as 1, 2, and 3 rather than starting at 0, which is required by the classifiers used in this study.

The global constants `RANDOM_STATE` and `CV_FOLDS` are also defined here, ensuring that all scripts use identical values for random seeding and cross-validation, which is essential for the comparability of results.

To read the datasets from their respective files, a custom `load_data` method was implemented. Depending on the dataset, the appropriate loading mechanism is selected. The *Wine* and *Credit* datasets are loaded using pandas' `read_csv` function. For the *Higgs* dataset, `read_parquet` is utilized. This was a deliberate decision to save storage space by storing the large *Higgs* dataset as a Parquet file rather than a standard CSV.

Following the initial loading phase, the *Higgs* data is reduced to a representative subset using a sampling approach controlled by the `nrows` parameter. Once the data size is managed, any columns specified in the configuration file are dropped from the respective dataset. The data is then split into the feature matrix X and the target vector y . If an offset is defined in the config, which is currently only the case for the *Credit* dataset, it is applied as described in the previous section. Finally, the method returns X and y for further use in the subsequent scripts. This process is visualized in Figure 5.1.

Due to the substantial scale of the *Higgs* dataset, several constraints were implemented to ensure the stability of the execution environment. Training a **Random Forest** model on this specific data led to unmanageable processing times. For this reason, the **Random Forest** algorithm is explicitly bypassed for the *Higgs* dataset and the program terminates using `sys.exit(0)` to prevent system crashes. This allows the process to stay within the boundaries of the available hardware resources.

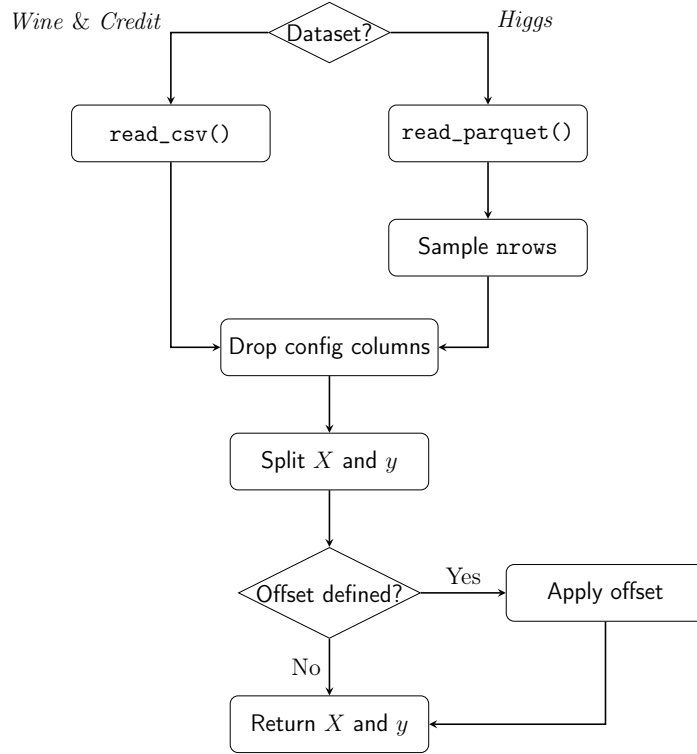


Figure 5.1: Visualization of the data loading pipeline and preprocessing steps within the `load_data()` method.

5.3 Model Implementations

As outlined in Section 4.3, the following classification algorithms were employed to address the research questions mentioned previously. This section therefore details the specific implementation of each model.

5.3.1 Logistic Regression

The first baseline model is a standard logistic regression. To ensure the model processes all features on an equal scale, the algorithm is embedded in a pipeline starting with a `StandardScaler`. This initial scaling step is crucial for the stability and overall performance of the subsequent classification.

The regression is configured to use the Stochastic Average Gradient (SAG) algorithm by setting the `solver` parameter to `sag`. This specific solver is highly optimized for large datasets. It significantly reduces the memory footprint and required training time, which is especially important when evaluating the extensive *Higgs* dataset. To ensure proper convergence, the maximum number of iterations is capped at 1000 using the `max_iter` parameter.

This model setup deliberately excludes any hyperparameter optimization. Instead, it acts as a stable baseline to evaluate the raw computational time without the extra overhead of complex search processes.

It is worth noting that the SAG solver is single-threaded and runs on a single CPU core, resulting in roughly 10% utilisation on the 8-core system used in this study. The `cross_validate()` call also uses no `n_jobs` parameter, so the five folds run sequentially. Each model in this study runs with its natural parallelisation behaviour; the implications of this for energy comparability are discussed in Section 5.5.2.

5.3.2 Random Forest

The second baseline algorithm is the Random Forest classifier. This method belongs to the family of ensemble learning techniques. Unlike logistic regression, this algorithm does not require prior feature scaling. To maximize computational efficiency, the execution is fully parallelized across all available CPU cores by setting the `n_jobs` parameter to `-1`. This configuration allows the processor to run at maximum capacity, which directly influences the subsequent energy consumption tracking and overall runtime measurements.

As established in Section 5.2, Random Forest is explicitly excluded from the *Higgs* dataset due to unmanageable training times at 11M instances. Including it would produce extreme runtimes without contributing meaningful insight into the scaling behaviour studied here.

To ensure maximum predictive capabilities for the remaining data, the model configurations were optimized using Optuna. The optimization process specifically targeted maximizing the weighted F1 score. For strict reproducibility, the final parameter combinations for the *Wine* and *Credit* datasets are documented in Table 5.2.

hier noch die korrekten werte nach dem finalen run eintragen

Table 5.2: Optimized hyperparameters for the Random Forest classifier obtained via Optuna optimization.

Parameter	<i>Wine</i>	<i>Credit</i>
<code>n_estimators</code>	221	381
<code>max_depth</code>	5	9
<code>min_samples_split</code>	8	10
<code>min_samples_leaf</code>	3	2
<code>max_features</code>	<code>sqrt</code>	<code>None</code>

5.3.3 XGBoost

The next tree-based ensemble model is the XGBoost classifier. This model serves as a direct point of comparison to the Random Forest. A primary reason for including XGBoost in this study is that it allows for a direct comparison of computational efficiency between CPU and GPU processing. To achieve this, the configuration dictionaries are kept identical for both executions. The only difference is the hardware device parameter, which is set to use the GPU. This controlled setup ensures a precise evaluation of the energy and emissions generated by the exact same model architecture on different hardware. This setup directly addresses the third research question: *How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*

The break-even analysis described in Section 4.1.1 is computed in the plotting pipeline by reading all HIGGS subset rows from `results.csv` and fitting the log-log model separately for CPU and GPU using `numpy.polyfit`. The intersection is calculated analytically using Equation (4.1).

Break-Even-Wert aus finalem Run eintragen und in Results referenzieren.

Additionally, XGBoost has a specific technical requirement for handling target variables. While the Random Forest algorithm natively adapts to different class structures, XGBoost requires explicit instructions for its learning objective. As mentioned in Section 4.2, the *Wine* dataset consists of three distinct categories. Therefore, the objective parameter is set to `multi:softmax`, paired with the `mlogloss` evaluation metric and a specified number of classes. In contrast, the *Credit* and *Higgs* datasets contain only two categories. Consequently, their objective is configured as `binary:logistic` and evaluated using the standard `logloss` metric.

Similar to the previous baseline algorithm, the parameters tailored to each dataset were determined using the Optuna optimization framework. Furthermore, the algorithm processes the unscaled feature matrix X directly. It also uses the globally defined random state parameter to guarantee a deterministic execution across all hardware setups. The resulting configurations used for the final training phase are detailed in Table 5.3.

Table 5.3: Optimized hyperparameters for the XGBoost classifier across all datasets.

Parameter	<i>Wine</i>	<i>Credit</i>	<i>Higgs</i>
objective	multi:softmax	binary:logistic	binary:logistic
eval_metric	mlogloss	logloss	logloss
num_class	3	N/A	N/A
n_estimators	464	108	475
max_depth	7	3	8
learning_rate	0.0139	0.2878	0.2428
subsample	0.7790	0.9640	0.7100
colsample_bytree	0.7500	0.8430	0.9368
min_child_weight	5	7	2

5.3.4 Multilayer Perceptron

As discussed in Section 4.3, a Multilayer Perceptron was selected to represent the neural network family. The implementation uses PyTorch as the primary deep-learning framework, combined with the Skorch library. Skorch wraps the PyTorch model in an interface that is fully compatible with the scikit-learn ecosystem. This is necessary because the entire evaluation logic, including cross-validation and pipeline construction, relies heavily on scikit-learn.

A dynamic architecture was chosen to ensure the network structure directly reflects the optimal parameters found during the Optuna tuning process. At runtime, the script reads the best parameter set from a JSON file generated during the tuning phase. It extracts the number of hidden layers via `n_layers` and the neuron count for each individual layer via `layer_i`. These values are then assembled into a list called `layer_sizes`. This list is subsequently passed to the `MLPModule` class. This class constructs the full network dynamically upon instantiation, making the architecture entirely data-driven.

The `MLPModule` inherits from `nn.Module` and builds the hidden layers by iterating over the layer size list. Each hidden block consists of `nn.Linear` followed by `nn.BatchNorm1d`, a ReLU activation, and a dropout layer (see Section 2.3.5 for the theoretical background on these components). After all hidden blocks, a single linear output layer maps to the number of target classes with no activation, since `CrossEntropyLoss` expects raw logits.

To minimize the overall training time, the implementation checks for CUDA availability using `torch.cuda.is_available()`. The algorithm runs on the GPU if one is present and automatically falls back to the CPU otherwise. PyTorch requires highly specific numeric types, and neural networks react sensitively to implicit type mismatches. Therefore, the feature matrix X is explicitly cast to `float32` and the target vector y to `int64` before the training begins.

The neural network is embedded within a scikit-learn `Pipeline` together with a

StandardScaler. Since Multilayer Perceptrons lack built-in scale invariance, standardizing the inputs to a mean of zero and a unit variance is necessary to keep the gradient flow stable. The **NeuralNetClassifier** provided by Skorch receives all architecture parameters through a specific naming convention. It uses the `module_*` prefix to forward keyword arguments directly to the constructor of the **MLPModule**. The Adam optimizer and the **CrossEntropyLoss** criterion are used consistently across all datasets. The learning rate and the batch size are taken directly from the tuning results and are therefore tailored to each dataset. A fixed random seed is applied using `torch.manual_seed` to guarantee exact reproducibility across all iterations.

The maximum number of training epochs is set to 200. This value acts as an upper boundary rather than a strict target. An early stopping callback (`skorch.callbacks.EarlyStopping`) monitors the validation loss after each individual epoch. It halts the training process immediately if no improvement is observed for 10 consecutive epochs. This mechanism prevents unnecessarily long training runs and provides an additional layer of protection against overfitting. Consequently, the upper limit of 200 epochs is rarely reached in practice.

5.4 Hyperparameter Optimization

Building on the design outlined in Section 4.6, this section describes how Optuna was integrated into the evaluation pipeline and which search spaces were defined for each model. The tuning logic is encapsulated in three dedicated scripts (`tune_xgb.py`, `tune_rfc.py`, and `tune_mlp.py`), one per tunable model. All three follow the same structural pattern and differ only in the model that is instantiated and the search space that is queried from the trial object. The target dataset is selected via a command-line argument, and the number of trials is read from the `N_TRIALS` environment variable, with a default of 50. This separation between configuration and code allows individual tuning runs to be triggered without modifying the source files. The search spaces used across all three tuning scripts are summarized in Table 5.4. Integer and continuous parameters define an inclusive range, while categorical parameters list the discrete choices available to the sampler.

After the dataset has been loaded through the shared `load_data()` function described in Section 5.2, each script constructs a `KFold` splitter using the same global constants `CV_FOLDS` and `RANDOM_STATE` that are also applied during the final evaluation. Reusing the identical splitter ensures that the F1 score returned to Optuna is computed under the same validation conditions as the metrics reported later in Chapter 6, eliminating any discrepancy between the score the optimizer maximizes and the score the study ultimately reports.

The core of every script is an `objective(trial)` function that draws a candidate hyperparameter configuration from the trial object, instantiates the corresponding clas-

Table 5.4: Hyperparameter search spaces defined in the Optuna tuning scripts.

Model	Parameter	Type	Range / Values
Random Forest	n_estimators	integer	[100, 500]
	max_depth	integer	[3, 20]
	min_samples_split	integer	[2, 20]
	min_samples_leaf	integer	[1, 10]
	max_features	categorical	{sqrt, log2, None}
XGBoost	n_estimators	integer	[100, 500]
	max_depth	integer	[3, 8]
	learning_rate	float (log)	[0.01, 0.3]
	subsample	float	[0.6, 1.0]
	colsample_bytree	float	[0.6, 1.0]
	min_child_weight	integer	[1, 10]
MLP	n_layers	integer	[1, 4]
	layer_i (per layer)	categorical	{64, 128, 256, 512}
	dropout_rate	float	[0.0, 0.5]
	lr	float (log)	[10 ⁻⁴ , 10 ⁻¹]
	batch_size	categorical	{1024, 4096, 8192}

sifier, and returns the mean weighted F1 score obtained from a five-fold cross-validation via `cross_val_score` with `make_scorer(f1_score, average="weighted")`. This is the only output that Optuna is exposed to. Everything else, such as emissions tracking and timing, is handled outside the objective function.

Each study is created via `optuna.create_study(direction="maximize")` and parameterized with a `TPESampler` seeded with `RANDOM_STATE`. Seeding the sampler is what makes the trial sequence reproducible across re-runs. Without it, the suggested hyperparameter combinations would differ on every execution, and the resulting tuning emissions would no longer be comparable. The optimization itself is launched through `study.optimize(objective, n_trials=N_TRIALS)`.

To capture the energy footprint of the tuning process itself, the entire `study.optimize` call is wrapped in a `CodeCarbon EmissionsTracker` with a script-specific `project_name` (e.g. `tune_mlp_higgs`), so that tuning runs are clearly distinguishable from training runs in the `emissions/` directory. The same tracker is used across all three scripts and is configured identically to the trackers used during training, as described in Section 5.5.

Once a study completes, the best score, the corresponding hyperparameter dictionary, the duration, the emissions reported by the tracker, and the trial count are written to a shared `best_params.json` file via the `save_best_params()` utility. The function performs an upsert keyed by model and dataset, so that re-running a single tuning configuration overwrites only its own entry and leaves the other results untouched. The model scripts introduced in Section 5.3 consume this file through the corresponding

`load_best_params()` helper, which decouples tuning from training entirely: the final training pipeline never instantiates Optuna, it merely reads the previously persisted hyperparameter dictionary and passes it to the classifier constructor.

5.5 Emissions Measurement

Measuring energy consumption and CO₂ emissions is the central objective of this study, and the measurement infrastructure therefore required particularly careful design. The following subsections describe how CodeCarbon was integrated into each script, how measurement coverage was defined across model and dataset combinations, and what hardware-specific constraints and limitations apply to the collected values.

5.5.1 CodeCarbon Integration

Energy consumption and CO₂ emissions were tracked using *CodeCarbon* as described in Section 2.2.3. In each model script, an `EmissionsTracker` instance is created before the cross-validation loop begins and configured with two key parameters: `output_dir`, which points to the shared `emissions/` directory at the project root, and `project_name`, which uniquely identifies the run. The tracker is then started by calling `tracker.start()` immediately before the `cross_validate()` call, and stopped via `tracker.stop()` once all folds have completed. The return value of `tracker.stop()` is a single float representing the total CO₂-equivalent emissions in kilograms accumulated over the entire tracked interval. This value is what gets passed to `save_results()` and written to `results.csv`.

The placement of the tracker boundaries was a deliberate choice. Data loading and preprocessing happen before `tracker.start()`, so they are excluded from the measurement. Only the cross-validation itself, which includes all five training and evaluation passes, falls inside the tracked interval. This ensures that the reported emissions value reflects the cost of model training and evaluation exclusively, and that values are directly comparable across models and datasets without interference.

Figure 5.2 illustrates this structure. Prior to training, data is loaded and, where required, features are normalized using `StandardScaler`. Scaling was applied to Logistic Regression and MLP, as these models are sensitive to the magnitude of input features, while tree-based models (Random Forest, XGBoost) were trained on the raw input data.

To allow fine-grained analysis, each model–dataset combination receives its own `EmissionsTracker` instance with a unique `project_name`. The naming convention follows the pattern `<model>_<dataset>`, for example `lr_wine`, `rf_credit`, or `xgb_gpu_higgs`. This scheme makes individual runs immediately identifiable in the `emissions/` directory, where CodeCarbon writes one CSV file per tracked run in addition to returning the

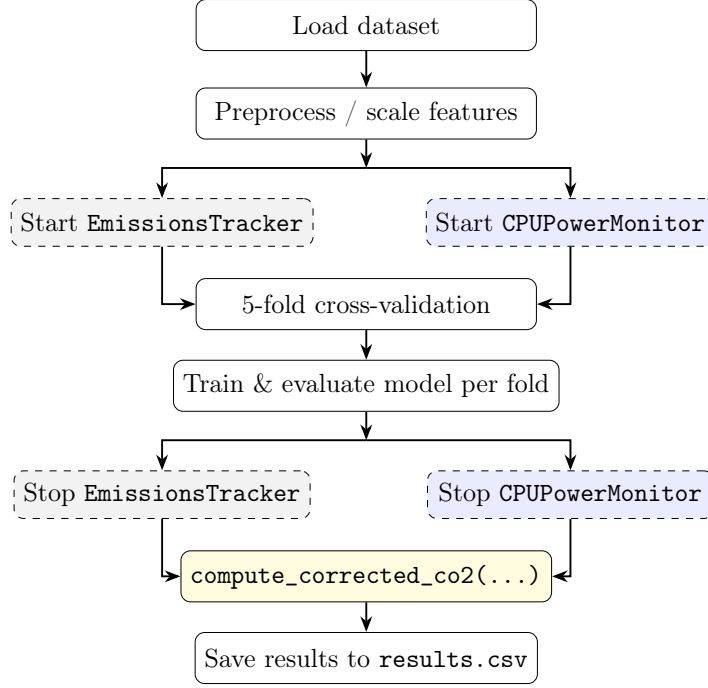


Figure 5.2: Training and emissions tracking procedure per model-dataset combination. Dashed gray boxes indicate CodeCarbon `EmissionsTracker` calls; dashed blue boxes indicate `CPUPowerMonitor` calls (background thread, 0.25s sampling). Both run in parallel during cross-validation. After training, `compute_corrected_co2` replaces CodeCarbon’s TDP-based CPU estimate with the hardware sensor value.

scalar value in code. Tuning scripts follow the same pattern preceded by `tune_`, for example `tune_mlp_higgs` or `tune_xgb_cpu_wine`. In total, the experiment produces tracked emissions records for 14 training runs and up to eight tuning runs. The separate CSV files serve as a raw audit trail, while the values extracted via `tracker.stop()` are the primary source for the analysis presented in Chapter 6.

5.5.2 Hardware Measurement and Limitations

The general measurement approach used by CodeCarbon, including direct GPU readout via NVML, RAM estimation per DIMM slot, and TDP-based CPU fallback on Windows, is described in Section 2.2.3. This subsection focuses on the concrete implications for the hardware used in this study.

CodeCarbon’s TDP-based CPU estimation on Windows has been replaced in this study by a custom `CPUPowerMonitor` class implemented in `models/power_monitor.py`. It runs in a background thread and polls the CPU Package Power sensor every 0.25 seconds via the *LibreHardwareMonitor* library (`HardwareMonitor` Python package). This library reads the CPU Package Power sensor directly from the hardware, providing true power values rather than a TDP approximation. In practice, CodeCarbon underestimated CPU power by a factor of roughly $9\times$ compared to the hardware sensor (e.g. 6.6 W vs. 57.8 W for Logistic Regression on the Credit dataset). The corrected

CO₂ value in `results.csv` therefore replaces CodeCarbon’s CPU contribution with the hardware-measured value, while GPU and RAM energy are still taken from CodeCarbon. The original CodeCarbon total is retained as `co2eq_codecarbon_kg` for reference.

This approach does, however, come with notable limitations. Because LibreHardwareMonitor requires privileged access to hardware sensors, all measurements in this study were conducted with the process running under administrator privileges. Without elevated rights, the `CPUPowerMonitor` fails to retrieve sensor data and the corrected CO₂ estimate cannot be computed.

A second limitation concerns the differences in CPU parallelisation across models. Logistic Regression, for instance, relies on the SAG solver, which executes on a single core, whereas Random Forest and XGBoost exploit multi-threading across all available threads. This disparity directly affects the instantaneous power draw: a single-threaded workload draws considerably fewer watts. However, the reduced wattage is largely offset by a proportionally longer runtime, meaning that the total energy in watt-hours may converge to a similar value. The implications of this trade-off for the comparability of emissions estimates are discussed in Chapter 7.

A secondary limitation is that CodeCarbon measures at the process level, meaning background system activity on Windows 11 contributes to the recorded energy. This effect is expected to be small relative to the dominant training workload but cannot be fully eliminated.

5.6 Inference Latency Measurement

Inference time was measured separately after training, outside the emissions tracker. For each model, a single sample from the test set was passed through the trained model and the elapsed time was recorded using `time.perf_counter()`. This provides a hardware-level estimate of per-sample prediction latency, which is relevant in deployment scenarios where repeated inference can accumulate significant energy costs [35].

Only a single row was passed to the model rather than a full batch. This was intentional: batching would measure throughput rather than the actual time it takes to process one request, which is the more relevant figure in deployment. A single-prediction latency can be extrapolated to real-world usage, where a model may handle millions of requests and the per-sample cost accumulates into a meaningful energy expenditure.

The model used for this measurement is taken from the first cross-validation fold via `cv_results["estimator"][0]`, which is made available by passing `return_estimator=True` to `cross_validate()`. The single-row slice `X[:1]` is then passed to `trained_model.predict()`, with the elapsed time captured using `time.perf_counter()` for sub-millisecond precision.

5.7 Results Persistence and Orchestration

This section describes how experimental results are stored and how the individual model scripts are coordinated into complete experiment runs. The two concerns are closely related: the storage format determines what data is available for analysis, while the orchestration layer controls how and in what order the model scripts are executed.

5.7.1 Results Schema

The results of every model run are written to `results/results.csv`, which serves as the central data source for all subsequent analysis and visualisation. Each row represents one complete model-dataset combination and is appended to the file rather than overwriting it, so that results from multiple runs accumulate without data loss. The file is managed using Python’s `csv.DictWriter`, which writes a header row on first creation and appends subsequent rows without repeating the header. A consequence of the append-only design is that re-running a configuration produces duplicate rows, which must be filtered during analysis. Table 5.5 lists all columns.

Table 5.5: Column schema of `results.csv`.

Column	Format	Description
<code>timestamp</code>	<code>datetime</code>	Date and time of the run
<code>model</code>	<code>string</code>	Model name
<code>dataset</code>	<code>string</code>	Dataset name
<code>nrows</code>	<code>int</code> / <code>all</code>	Subset size; <code>all</code> for full dataset
<code>accuracy</code>	<code>float</code>	Mean CV accuracy
<code>f1</code>	<code>float</code>	Mean weighted F1 score
<code>co2eq_kg</code>	<code>float</code>	Corrected CO ₂
<code>co2eq_codecarbon_kg</code>	<code>float</code>	Original CodeCarbon estimate
<code>cpu_power_hw_w</code>	<code>float</code>	Average CPU package power (W)
<code>cpu_energy_hw_wh</code>	<code>float</code>	Total CPU energy (Wh)
<code>training_time_s</code>	<code>float</code>	Total CV wall-clock time (s)

The primary emissions value is `co2eq_kg`, which combines the hardware-sensor CPU reading with CodeCarbon’s GPU and RAM estimates as described in Section 5.5.2. The column `co2eq_codecarbon_kg` retains the original CodeCarbon total for reference. The difference between the two columns is examined in Chapter 7.

Inference latency is stored separately in `results/inference_time.csv`, as it is measured outside the emissions tracker and has a different structure. Like `results.csv`, it uses append-only writes via `csv.DictWriter`. Table 5.6 shows its columns.

5.7.2 Experimental Orchestration

Three orchestration scripts coordinate the full experiment. Each model script is launched as an isolated subprocess rather than being imported directly. This is important because

Table 5.6: Column schema of `inference_time.csv`.

Column	Format	Description
<code>timestamp</code>	<code>datetime</code>	Date and time of the run
<code>model</code>	<code>string</code>	Model name
<code>dataset</code>	<code>string</code>	Dataset name
<code>nrows</code>	<code>int / all</code>	Subset size; <code>all</code> for full dataset
<code>inference_time</code>	<code>float</code>	Single-row prediction latency (s)
<code>co2eq_kg</code>	<code>float</code>	Corrected CO ₂ of the training run

CodeCarbon operates at the process level: running multiple trackers within the same process risks state leakage between measurements. Subprocess isolation ensures that each tracker starts and stops cleanly. A failed script does not abort the overall run; failures are collected and reported at the end.

`run_all.py` is the main entry point for the full experiment. It runs in two sequential phases. Phase 1 executes all tuning scripts for Random Forest, XGBoost, and MLP across their respective datasets, and saves the best parameters to `best_params.json`. Phase 2 then runs all five model scripts across all three datasets. Each script is launched via `subprocess.run()` with the working directory set to `models/`, so that relative imports resolve correctly.

The other two scripts are used for the subset-based analyses. `run_xgb_breakeven` runs XGBoost CPU and GPU directly on nine HIGGS subset sizes ranging from 1,000 to 11,000,000 rows. Unlike the other orchestration scripts, it calls the model logic directly rather than via subprocess, because it manages the emissions tracker and the cross-validation loop itself. This script provides the data for the break-even analysis in Chapter 6. `run_scaling_all_models.py` runs all models except Random Forest via subprocess across five HIGGS subset sizes (100k to 11M rows), and provides the data for the scaling analysis.

Both subset scripts control the dataset size through an environment variable `TEST_NROWS`. When set, `load_data()` reads this value and draws a stratified sample of that size instead of loading the full dataset. The value is written to the `nrows` column in `results.csv` as a concrete integer, making subset results directly filterable during analysis. The key advantage of this approach is that no changes to any model script are required; the subset size is controlled entirely from the outside.

Chapter 6

Results and Evaluation

Chapter 7

Discussion

Zur Aussagekraft des Break-Even-Points: Der log-log-Fit basiert auf 9 HIGGS-Subset-Größen (1k–11M) mit gleichen Hyperparametern — das isoliert den Dataset-Größen-Effekt sauber. Dennoch Limitierungen benennen: (1) Potenzgesetz ist eine Approximation. (2) Ergebnis gilt nur für diese Hardware und diesen Datensatz. (3) Fit empfindlich gegenüber Ausreißern an den Rändern. Break-Even-Wert als Größenordnung interpretieren, nicht als präzisen Schwellenwert.

CodeCarbon vs. HardwareMonitor comparison: discuss the systematic underestimation of CPU power by CodeCarbon on Windows (TDP-based fallback vs. sensor readout). Show the difference between `co2eq_codecarbon_kg` and `co2eq_kg` per model/dataset. Interpret what this means for reproducibility and comparability of emissions studies that rely solely on CodeCarbon.

Parallelisation trade-off: Logistic Regression (SAG solver, single core) draws fewer watts than Random Forest / XGBoost (multi-threaded), but the longer runtime may result in comparable total energy in Wh. Discuss whether watt-based or Wh-based comparison is the fairer metric for cross-model emissions comparability, and what this means for interpreting the results.

Chapter 8

Conclusion and Outlook

Bibliography

- [1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.
- [2] E. Strubell, A. Ganesh, and A. McCallum, *Energy and Policy Considerations for Deep Learning in NLP*, Jun. 2019. DOI: 10.48550/arXiv.1906.02243 arXiv: 1906.02243 [cs]. Accessed: Mar. 19, 2026.
- [3] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.
- [4] R. Verdecchia, L. Cruz, J. Sallou, M. Lin, J. Wickenden, and E. Hotellier, “Data-Centric Green AI An Exploratory Empirical Study,” in *2022 International Conference on ICT for Sustainability (ICT4S)*, Plovdiv, Bulgaria: IEEE, Jun. 2022, pp. 35–45, ISBN: 978-1-6654-8286-8. DOI: 10.1109/ICT4S55073.2022.00015 Accessed: Mar. 26, 2026.
- [5] *AI and compute*, <https://openai.com/index/ai-and-compute/>, Jun. 2022. Accessed: Apr. 19, 2026.
- [6] *Green AI Position Paper*, <https://greensoftware.foundation/policy/research/green-ai-position-paper/>. Accessed: Mar. 30, 2026.
- [7] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, *Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning*, Nov. 2022. DOI: 10.48550/arXiv.2002.05651 arXiv: 2002.05651 [cs]. Accessed: Mar. 19, 2026.
- [8] T. Eilam, P. Bello-Maldonado, B. Bhattacharjee, C. Costa, E. K. Lee, and A. Tantawi, “Towards a Methodology and Framework for AI Sustainability Metrics,” in *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, ser. HotCarbon ’23, New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 1–7, ISBN: 979-8-4007-0242-6. DOI: 10.1145/3604930.3605715 Accessed: Apr. 10, 2026.

- [9] J. Dodge et al., “Measuring the Carbon Intensity of AI in Cloud Instances,” in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*, ser. FAccT ’22, New York, NY, USA: Association for Computing Machinery, Jun. 2022, pp. 1877–1894, ISBN: 978-1-4503-9352-2. DOI: 10.1145/3531146.3533234 Accessed: Mar. 11, 2026.
- [10] C. Rodriguez, L. Degioanni, L. Kameni, R. Vidal, and G. Neglia, *Evaluating the Energy Consumption of Machine Learning: Systematic Literature Review and Experiments*, Aug. 2024. DOI: 10.48550/arXiv.2408.15128 arXiv: 2408.15128 [cs]. Accessed: Mar. 11, 2026.
- [11] N. Bannour, S. Ghannay, A. Név  ol, and A.-L. Ligozat, “Evaluating the carbon footprint of NLP methods: A survey and analysis of existing tools,” in *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, Virtual: Association for Computational Linguistics, 2021, pp. 11–21. DOI: 10.18653/v1/2021.sustainlp-1.2 Accessed: Mar. 26, 2026.
- [12] R. Bommasani et al., *On the Opportunities and Risks of Foundation Models*, Jul. 2022. DOI: 10.48550/arXiv.2108.07258 arXiv: 2108.07258 [cs]. Accessed: Mar. 11, 2026.
- [13] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” in *Proceedings of the 35th International Conference on Neural Information Processing Systems*, ser. NIPS ’21, Red Hook, NY, USA: Curran Associates Inc., Dec. 2021, pp. 18 932–18 943, ISBN: 978-1-7138-4539-3. Accessed: Mar. 26, 2026.
- [14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, ISSN: 1533-7928. Accessed: Apr. 18, 2026.
- [15] J. H. Friedman, “Greedy function approximation: A gradient boosting machine.,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, ISSN: 0090-5364, 2168-8966. DOI: 10.1214/aos/1013203451 Accessed: Apr. 19, 2026.
- [16] D. R. Cox, “The Regression Analysis of Binary Sequences,” *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.
- [17] J. S. Cramer, *The Origins of Logistic Regression*, SSRN Scholarly Paper, Rochester, NY, Dec. 2002. DOI: 10.2139/ssrn.360300 Social Science Research Network: 360300. Accessed: Apr. 28, 2026.
- [18] L. Grinsztajn, E. Oyallon, and G. Varoquaux, *Why do tree-based models still outperform deep learning on tabular data?* Jul. 2022. DOI: 10.48550/arXiv.2207.08815 arXiv: 2207.08815 [cs]. Accessed: Mar. 19, 2026.

- [19] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.
- [20] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.
- [21] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, ISSN: 1939-1471. DOI: 10.1037/h0042519
- [22] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Apr. 19, 2026.
- [23] G. Hinton, V. Nair, and Y. Rachmad, “Rectified Linear Units Improve Restricted Boltzmann Machines Vinod Nair,” vol. 27, pp. 807–814, Jun. 2010.
- [24] D. P. Kingma and L. J. Ba, “Adam: A Method for Stochastic Optimization,” 2015. Accessed: Apr. 19, 2026.
- [25] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, Jun. 2015, pp. 448–456. Accessed: Apr. 18, 2026.
- [26] A. Althnian et al., “Impact of Dataset Size on Classification Performance: An Empirical Evaluation in the Medical Domain,” *Applied Sciences*, vol. 11, no. 2, p. 796, Jan. 2021, ISSN: 2076-3417. DOI: 10.3390/app11020796 Accessed: Apr. 16, 2026.
- [27] T. Hastie, R. Tibshirani, and J. Friedman, “Model Assessment and Selection,” in *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, T. Hastie, R. Tibshirani, and J. Friedman, Eds., New York, NY: Springer, 2009, pp. 219–259, ISBN: 978-0-387-84858-7. DOI: 10.1007/978-0-387-84858-7_7 Accessed: Apr. 10, 2026.
- [28] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, ISSN: 1533-7928. Accessed: Apr. 19, 2026.
- [29] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for Hyper-Parameter Optimization,” in *Advances in Neural Information Processing Systems*, vol. 24, Curran Associates, Inc., 2011. Accessed: Apr. 19, 2026.

- [30] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD ’19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.
- [31] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: 10.24432/C5PC7J Accessed: Mar. 19, 2026.
- [32] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: 10.24432/C55S3H Accessed: Mar. 19, 2026.
- [33] P. Baldi, P. Sadowski, and D. Whiteson, “Searching for Exotic Particles in High-Energy Physics with Deep Learning,” *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: 10.1038/ncomms5308 arXiv: 1402.4735 [hep-ph]. Accessed: Mar. 19, 2026.
- [34] D. Whiteson, *HIGGS*, 2014. DOI: 10.24432/C5V312 Accessed: Mar. 19, 2026.
- [35] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, “Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning,” *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.
- [36] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: 10.48550/arXiv.1703.06777 arXiv: 1703.06777 [cs]. Accessed: Apr. 10, 2026.
- [37] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.
- [38] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: 10.48550/arXiv.1912.01703 arXiv: 1912.01703 [cs]. Accessed: Mar. 19, 2026.
- [39] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.
- [40] B. Courty et al., *Mlco2/codecarbon: V2.4.1*, Zenodo, May 2024. DOI: 10.5281/zenodo.11171501 Accessed: Apr. 15, 2026.

List of Figures

5.1	Visualization of the data loading pipeline and preprocessing steps within the <code>load_data()</code> method.	29
5.2	Training and emissions tracking procedure per model-dataset combination. Dashed gray boxes indicate CodeCarbon <code>EmissionsTracker</code> calls; dashed blue boxes indicate <code>CPUPowerMonitor</code> calls (background thread, 0.25 s sampling). Both run in parallel during cross-validation. After training, <code>compute_corrected_co2</code> replaces CodeCarbon’s TDP-based CPU estimate with the hardware sensor value.	36

List of Tables

4.1	Overview of datasets used in this analysis	22
4.2	Overview of classification models used in this study	23
5.1	Hardware specification of the experimental system	27
5.2	Optimized hyperparameters for the Random Forest classifier obtained via Optuna optimization.	31
5.3	Optimized hyperparameters for the XGBoost classifier across all datasets.	32
5.4	Hyperparameter search spaces defined in the Optuna tuning scripts. . .	34
5.5	Column schema of <code>results.csv</code>	38
5.6	Column schema of <code>inference_time.csv</code>	39